

SBTTD Data Analysis Code

Developer & User Documentation

Supersonic Bladeless Turbine Theoretical Development

Hadie Sabbah

Last Updated: May 3, 2026

Contents

1	Introduction	2
1.1	Repository Structure	2
1.2	Conventions Used in This Document	2
2	utils/experimental.py — Experimental Data Processing	3
2.1	Design Philosophy	3
2.2	Supported Pressure Units	3
2.3	_convert_pressure(value_pa, target_unit)	3
2.4	set_publication_style(dpi=300)	4
2.5	exp_cleaner(dirc)	4
2.6	detect_test_window(clean_data, k_sigma=10, n_baseline=50)	5
2.7	exp_plotter(x_data, y_dict, y_name, label_name, y_unit, show=True)	5
2.8	exp_stats_pressure(clean_data, confidence=0.95)	5
2.9	slice_extractor_wall(dirc)	6
2.10	process_exp_case(...)	6
2.11	plot_exp_errorbar(exp_result, ...)	8
2.12	compare_exp_to_cfd(...)	8
3	Complete Workflow Example	11
4	How to Add a New Pressure Unit	12
5	sweep_study_gui.py — Desktop GUI Application	13
5.1	Prerequisites	13
5.2	Launching the GUI	13
5.3	GUI Architecture	13
5.4	Application Structure	13
5.5	How to Extend the GUI	14
5.6	User Workflow	15

Section 1: Introduction

This document provides comprehensive documentation for the SBTTD data analysis codebase. It is intended to serve as both a developer reference and a user guide so that lab members can understand and use the post-processing pipeline without needing to read the source code directly.

1.1 Repository Structure

The relevant code lives under:

```
2_SBTDD Data Analysis Code/
  MachPostProcessCode.py      # Main analysis script
  utils/
    __init__.py
    experimental.py           # Experimental + CFD comparison functions
    dataload_util.py          # Tecplot data import / save / load
    parameterComputation.py    # Variable extraction & Reynolds number
    computation.py             # (placeholder)
    models.py                  # Theoretical models (SPT, SE, geometry)
    plotting.py                # Plotting utilities
  docs/
    sbttdd_code_documentation.tex  # This document
```

1.2 Conventions Used in This Document

- Function signatures are shown in monospace.
- Parameters marked with a default value (e.g. `=1.0`) are optional.
- All code examples assume the working directory is `2_SBTDD Data Analysis Code/`.

Section 2: `utils/experimental.py` — Experimental Data Processing

This module provides a complete pipeline for loading wind-tunnel experimental CSV data, auto-detecting test windows, computing statistics, and comparing experimental results against CFD solutions extracted from Tecplot. All functions support automatic pressure unit conversion so that experimental and CFD data can be brought into a common unit system before comparison.

2.1 Design Philosophy

The module is designed around two high-level workflow functions (`process_exp_case` and `compare_exp_to_cfd`) that chain together lower-level helpers. Each function can also be called independently. The conversion path is:

$$\text{raw value} \xrightarrow{\times \text{pressure_scale} + \text{pressure_offset}} \text{native unit} \xrightarrow{\div \text{factor}} \text{Pa} \xrightarrow{\times \text{factor}} \text{target unit} \quad (1)$$

This two-hop design (always passing through Pa as the canonical intermediate) guarantees that any source unit can be converted to any target unit without the user needing to know pairwise conversion factors.

2.2 Supported Pressure Units

The module stores conversion factors in a dictionary `PRESSURE_UNIT_FROM_PA`. To convert a value from Pa to the target unit, multiply by the corresponding factor.

Table 1: Supported pressure units and their conversion factors from Pa.

Unit key	Description	Factor (from Pa)
'Pa'	Pascal	1.0
'kPa'	Kilopascal	10^{-3}
'MPa'	Megapascal	10^{-6}
'bar'	Bar	10^{-5}
'atm'	Standard atmosphere	1/101325
'psi'	Pounds per sq. inch	1/6894.757

To add a new unit, append one entry to `PRESSURE_UNIT_FROM_PA` and one to `PRESSURE_UNIT_LABELS` at the top of `experimental.py`. No other code changes are needed.

2.3 `_convert_pressure(value_pa, target_unit)`

Purpose: Internal helper that converts a pressure value (scalar or NumPy array) from Pa to the specified target unit.

Table 2: Parameters for `_convert_pressure`.

Parameter	Type	Description
<code>value_pa</code>	float or <code>np.array</code>	Pressure in Pa
<code>target_unit</code>	str	Key from Table 1

Returns: The converted value in the target unit.

Raises: `ValueError` if `target_unit` is not a recognized key.

2.4 set_publication_style(dpi=300)

Purpose: Configure matplotlib rcParams for publication-quality figures (Times New Roman, appropriate font sizes, line widths). Call this *once* at the start of your script before any plotting.

Table 3: Parameters for set_publication_style.

Parameter	Type	Default	Description
dpi	int	300	Resolution for display and saved figures

Returns: None (modifies global matplotlib state).

```
1 from utils.experimental import set_publication_style
2 set_publication_style(dpi=600) # higher res for journal submission
```

Listing 1: Setting publication style.

2.5 exp_cleaner(dirc)

Purpose: Read an experimental CSV file produced by the data acquisition system and return a structured dictionary with sensor data separated by type (pressure vs. temperature), keyed by channel number.

The function auto-detects sensor columns using the regex pattern `(SN\d+)-([PT])(\d+)`, which matches headers like SN1051-P01, SN1051-T06, etc. Any column that does not match this pattern is treated as meta-data (e.g. Frame, FTime[s], FTime[ns]).

Table 4: Parameters for exp_cleaner.

Parameter	Type	Description
dirc	str or Path	Path to the CSV file

Returns: A dictionary with four keys:

Table 5: Return structure for exp_cleaner.

Key	Description
'sensor_map'	dict — Full column metadata: {col_idx: {'serial', 'type', 'channel', 'header'}}
'meta'	dict — Non-sensor columns: {header_name: np.array}
'P'	dict — Pressure data: {channel_number: np.array}
'T'	dict — Temperature data: {channel_number: np.array}

```
1 from utils.experimental import exp_cleaner
2 data = exp_cleaner(r"C:\...\Scan.csv")
3 print(data['P'].keys()) # e.g. dict_keys([1, 2, 3, ...])
4 print(data['meta'].keys()) # e.g. dict_keys(['Frame', 'FTime[s]', ...])
```

Listing 2: Basic usage of exp_cleaner.

2.6 detect_test_window(clean_data, k_sigma=10, n_baseline=50)

Purpose: Automatically detect the start and end of the wind-tunnel test run from the pressure time series. The algorithm:

1. Identifies the stagnation channel (highest mean pressure).
2. Estimates the noise floor from the first `n_baseline` frames.
3. Sets a threshold at $\mu_{\text{baseline}} + k_{\sigma} \cdot \sigma_{\text{baseline}}$.
4. Returns the first and last frame indices where pressure exceeds this threshold.

Table 6: Parameters for `detect_test_window`.

Parameter	Type	Default	Description
<code>clean_data</code>	dict	—	Output from <code>exp_cleaner()</code>
<code>k_sigma</code>	float	10	Threshold in standard deviations
<code>n_baseline</code>	int	50	Number of pre-test frames for noise estimation

Returns: (`i_start`, `i_end`, `t_start`, `t_end`) — frame indices and corresponding times in seconds.

Raises: `ValueError` if no frames exceed the threshold.

2.7 exp_plotter(x_data, y_dict, y_name, label_name, y_unit, show=True)

Purpose: Plot time-series data for each sensor channel on a single axes. Primarily used to visualize raw pressure or temperature traces during a test run.

Table 7: Parameters for `exp_plotter`.

Parameter	Type	Default	Description
<code>x_data</code>	<code>np.array</code>	—	Time array (x-axis)
<code>y_dict</code>	dict	—	{ <code>channel</code> : <code>np.array</code> }
<code>y_name</code>	str	—	Y-axis label name (e.g. “Pressure”)
<code>label_name</code>	str	—	Legend prefix (e.g. “P” or “T”)
<code>y_unit</code>	str	—	Unit string (e.g. “Pa”)
<code>show</code>	bool	True	Whether to call <code>plt.show()</code>

Returns: (`fig`, `ax`) — Matplotlib figure and axes objects for further customization.

2.8 exp_stats_pressure(clean_data, confidence=0.95)

Purpose: Compute descriptive statistics for each pressure channel over the auto-detected test window. Internally calls `detect_test_window()` to crop the data.

Statistics computed for each channel:

- $\bar{x}$ (mean)
- s (standard deviation, with Bessel’s correction $\text{ddof} = 1$)
- $\text{SEM} = s/\sqrt{N}$
- 95% confidence interval: $\text{CI} = t_{\alpha/2, N-1} \cdot \text{SEM}$

Returns: dict — {`channel`: {`’mean’`, `’std’`, `’sem’`, `’CI_95’`, `’N’`}}.

Note: the returned values are in the *raw sensor units* (whatever the CSV contains). Unit conversion happens downstream in `process_exp_case`.

Table 8: Parameters for `exp_stats_pressure`.

Parameter	Type	Default	Description
<code>clean_data</code>	dict	—	Output from <code>exp_cleaner()</code>
<code>confidence</code>	float	0.95	Confidence level for CI

2.9 `slice_extractor_wall(dirc)`

Purpose: Extract the bottom-wall surface data from a 3D Tecplot CFD solution by slicing at $z = 0$. Requires an active Tecplot session (`tp.session.connect()`).

Algorithm:

1. Load the Tecplot binary file (or reuse if already loaded).
2. Set the frame to Cartesian 3D.
3. Extract a slice at $z = 0$ from the “Section” surface zone, using `OneZonePerConnectedRegion` mode.
4. Select the zone with the lowest mean Y (the bottom wall).
5. Sort all arrays by X and return.

Table 9: Parameters for `slice_extractor_wall`.

Parameter	Type	Description
<code>dirc</code>	str or Path	Path to <code>mcfcd_tec.bin</code>

Returns: (`x_wall`, `y_wall`, `p_wall`, `T_wall`) — all `np.array`, sorted by x . Pressures are in the CFD solver’s native unit (typically Pa).

2.10 `process_exp_case(...)`

Purpose: One-call pipeline that loads a CSV, computes elapsed time, converts pressures, runs statistics, and returns everything in a single consistent pressure unit. This is the primary entry point for processing an experimental dataset.

Full signature:

```
process_exp_case(
    csv_path,
    tap_spacing_mm,
    skip_channels=0,
    pressure_scale=1.0,
    pressure_offset=0.0,
    exp_native_unit='Pa',
    pressure_unit='Pa',
    plot=True,
)
```

Table 10: Parameters for `process_exp_case`.

Parameter	Type	Default	Description
<code>csv_path</code>	str/Path	—	Path to experimental CSV

Parameter	Type	Default	Description
tap_spacing_mm	float	—	Physical distance between taps [mm]
skip_channels	int	0	Leading channels to exclude from stats
pressure_scale	float	1.0	Multiplier: raw $\rightarrow$ exp_native_unit
pressure_offset	float	0.0	Offset added after scaling (in native unit)
exp_native_unit	str	'Pa'	Unit produced by scale + offset
pressure_unit	str	'Pa'	Desired output unit (see Table 1)
plot	bool	True	Show time-series plots

Unit conversion logic (see also Eq. 1):

1. Multiply raw sensor reading by `pressure_scale`.
2. Add `pressure_offset`. The result is now in `exp_native_unit`.
3. Convert to Pa: divide by `PRESSURE_UNIT_FROM_PA[exp_native_unit]`.
4. Convert Pa to `pressure_unit`: multiply by `PRESSURE_UNIT_FROM_PA[pressure_unit]`.

Returns: A dictionary with the following keys:

Table 11: Return structure for `process_exp_case`.

Key	Description
'clean_data'	Raw output from <code>exp_cleaner()</code>
't_elapsed'	Elapsed time array [s]
'P_converted'	{channel: np.array} in <code>pressure_unit</code>
'T_data'	{channel: np.array} temperature data
'stats'	Raw statistics from <code>exp_stats_pressure()</code>
'channels'	Sorted channel list (after skipping)
'means'	np.array of mean pressures in <code>pressure_unit</code>
'errors'	np.array of 95% CI values in <code>pressure_unit</code>
'x_exp_mm'	Tap locations in mm
'pressure_unit'	String recording the output unit

Example: Flat Plate (raw in bar, output in kPa)

```

1 from utils.experimental import set_publication_style, process_exp_case
2
3 set_publication_style()
4
5 flat = process_exp_case(
6     csv_path=r"C:\...\Scan.csv",
7     tap_spacing_mm=7.25,
8     skip_channels=3,           # skip stagnation ports
9     pressure_scale=1e5,       # raw is in bar -> * 1e5 = Pa
10    pressure_offset=101325,    # gauge -> absolute
11    exp_native_unit='Pa',      # after scale+offset we have Pa
12    pressure_unit='kPa',       # output everything in kPa
13 )
14 print(flat['means'])          # array in kPa
15 print(flat['pressure_unit'])  # 'kPa'

```

Listing 3: Processing a flat-plate experiment.

Example: $h/l = 0.02$ Angled (raw in atm, output in bar)

```

1 angled = process_exp_case(
2     csv_path=r"C:\...\Test1.csv",
3     tap_spacing_mm=5.65,
4     skip_channels=3,
5     pressure_scale=101325, # raw is in atm -> * 101325 = Pa
6     exp_native_unit='Pa',
7     pressure_unit='bar',   # output in bar
8 )

```

Listing 4: Processing an angled-geometry experiment.

2.11 plot_exp_errorbar(exp_result, ...)

Purpose: Generate an error-bar plot of mean wall pressure vs. tap location from the output of `process_exp_case`. The y-axis label is auto-generated from the stored `pressure_unit` unless overridden.

Table 12: Parameters for `plot_exp_errorbar`.

Parameter	Type	Default	Description
<code>exp_result</code>	dict	—	Output from <code>process_exp_case()</code>
<code>title</code>	str	"Experimental Wall Pressure"	Plot title
<code>xlabel</code>	str	"X [mm]"	X-axis label
<code>ylabel</code>	str	None	Auto from unit if None
<code>show</code>	bool	True	Call <code>plt.show()</code>

Returns: (fig, ax).

```

1 from utils.experimental import plot_exp_errorbar
2
3 fig, ax = plot_exp_errorbar(flat, title="Flat Plate")
4 # ylabel will automatically be "Pressure [kPa]"

```

Listing 5: Plotting experimental error bars.

2.12 compare_exp_to_cfd(...)

Purpose: Load CFD wall data from Tecplot, trim to a matching x-range, convert both CFD and experimental pressures to a common unit, and produce an overlay plot. This function guarantees unit consistency between the two data sources.

Full signature:

```

compare_exp_to_cfd(
    exp_result, cfd_path, x_min_cfd, x_max_cfd,

```



```

    cfd_native_unit='Pa',
    cfd_pressure_offset=0.0,
    pressure_unit=None,
    normalize=False,
    title="Pressure Vs X",
    xlabel="X [mm]",
    ylabel=None,
    show=True,
)

```

Table 13: Parameters for compare_exp_to_cfd.

Parameter	Type	Default	Description
exp_result	dict	—	Output from process_exp_case()
cfd_path	str/Path	—	Path to mcftec.bin
x_min_cfd	float	—	Left trim bound (CFD coords)
x_max_cfd	float	—	Right trim bound (CFD coords)
cfd_native_unit	str	'Pa'	Unit the CFD solver writes
cfd_pressure_offset	float	0.0	Subtracted from CFD P before conversion (in native unit)
pressure_unit	str/None	None	Target unit; if None, matches experiment
normalize	bool	False	Normalize both by their maxima
title	str	—	Plot title
xlabel	str	—	X-axis label
ylabel	str/None	None	Auto from unit if None
show	bool	True	Call plt.show()

Unit matching logic:

- If pressure_unit=None: the function reads exp_result['pressure_unit'] and converts CFD data to that same unit. **No mismatch is possible.**
- If pressure_unit='psi' (or any valid key): both CFD and experimental data are converted to that unit, regardless of what either started in.
- The CFD conversion path is: $P_{\text{CFD}} - \text{offset} \rightarrow \text{Pa} \rightarrow \text{pressure_unit}$.
- If the experiment was already processed in a different unit, it is re-converted: $\text{exp_unit} \rightarrow \text{Pa} \rightarrow \text{pressure_unit}$.

Returns: A dictionary:

Table 14: Return structure for compare_exp_to_cfd.

Key	Description
'x_cfd', 'P_cfd'	Trimmed CFD arrays in pressure_unit
'x_exp', 'P_exp', 'errors'	Experimental arrays in pressure_unit
'x_wall', 'y_wall', 'P_wall', 'T_wall'	Raw wall data (CFD native unit)
'pressure_unit'	The unit used for comparison
'fig', 'ax'	Matplotlib figure and axes

Example: Auto-match experiment unit

```
1 from utils.experimental import compare_exp_to_cfd
2
3 # flat was processed with pressure_unit='kPa'
4 result = compare_exp_to_cfd(
5     flat,
6     cfd_path=r"C:\...\mcftec.bin",
7     x_min_cfd=40, x_max_cfd=127,
8     cfd_native_unit='Pa',
9     cfd_pressure_offset=94748,
10    # pressure_unit=None -> auto uses 'kPa' from flat
11 )
12 # Both curves are in kPa, ylabel = "P [kPa]"
```

Listing 6: CFD vs. experiment — auto unit matching.

Example: Force a different unit

```
1 result = compare_exp_to_cfd(
2     flat,
3     cfd_path=r"C:\...\mcftec.bin",
4     x_min_cfd=40, x_max_cfd=127,
5     cfd_native_unit='Pa',
6     cfd_pressure_offset=94748,
7     pressure_unit='psi', # forces both to psi
8 )
9 # ylabel = "P [psi]"
```

Listing 7: Override to compare in psi.

Example: Normalized comparison

```
1 result = compare_exp_to_cfd(
2     flat,
3     cfd_path=r"C:\...\mcftec.bin",
4     x_min_cfd=40, x_max_cfd=127,
5     cfd_native_unit='Pa',
6     cfd_pressure_offset=94748,
7     normalize=True,
8 )
9 # ylabel = "Normalized P [-]"
```

Listing 8: Normalized overlay.

Section 3: Complete Workflow Example

The following script demonstrates the full pipeline for processing three experimental cases and comparing each to its corresponding CFD solution, all in a common pressure unit.

```
1 from pathlib import Path
2 from utils.experimental import (
3     set_publication_style,
4     process_exp_case,
5     plot_exp_errorbar,
6     compare_exp_to_cfd,
7 )
8
9 set_publication_style()
10
11 # -- 1. Flat Plate -----
12 flat = process_exp_case(
13     csv_path=r"C:\...\Scan.csv",
14     tap_spacing_mm=7.25,
15     skip_channels=3,
16     pressure_scale=1e5,
17     pressure_offset=101325,
18     exp_native_unit='Pa',
19     pressure_unit='kPa',
20 )
21 plot_exp_errorbar(flat, title="Flat Plate")
22
23 compare_exp_to_cfd(
24     flat,
25     cfd_path=r"C:\...\mcftec.bin",
26     x_min_cfd=40, x_max_cfd=127,
27     cfd_native_unit='Pa',
28     cfd_pressure_offset=94748,
29 )
30
31 # -- 2. h/l = 0.02 Angled -----
32 angled = process_exp_case(
33     csv_path=r"C:\...\Test1.csv",
34     tap_spacing_mm=5.65,
35     skip_channels=3,
36     pressure_scale=101325,
37     exp_native_unit='Pa',
38     pressure_unit='kPa',
39 )
40 plot_exp_errorbar(angled, title="h/l = 0.02 Angled")
41
42 compare_exp_to_cfd(
43     angled,
44     cfd_path=r"C:\...\mcftec.bin",
45     x_min_cfd=-0.005, x_max_cfd=0.0678,
46     cfd_native_unit='Pa',
47     cfd_pressure_offset=101325,
48 )
49
```

```
50 # -- 3. h/l = 0.02 Straight -----
51 straight = process_exp_case(
52     csv_path=r"C:\...\P_tap_results_Test8.csv",
53     tap_spacing_mm=7.5,
54     skip_channels=1,
55     pressure_scale=101325,
56     exp_native_unit='Pa',
57     pressure_unit='kPa',
58 )
59 plot_exp_errorbar(straight, title="h/l = 0.02 Straight")
```

Listing 9: Full workflow for three experimental cases.

Section 4: How to Add a New Pressure Unit

To add support for a new pressure unit (e.g. mmHg), edit the two dictionaries at the top of `utils/experimental.py`:

```
1 PRESSURE_UNIT_FROM_PA = {
2     'Pa': 1.0,
3     'kPa': 1e-3,
4     # ... existing entries ...
5     'mmHg': 1.0 / 133.322, # <-- add this line
6 }
7
8 PRESSURE_UNIT_LABELS = {
9     'Pa': 'Pa',
10    # ... existing entries ...
11    'mmHg': 'mmHg', # <-- add this line
12 }
```

Listing 10: Adding mmHg support.

No other code changes are required. The new unit will work with all functions immediately.

Section 5: sweep_study_gui.py — Desktop GUI Application

This section documents the graphical user interface (GUI) application that wraps the analysis pipeline into a point-and-click tool for lab members.

5.1 Prerequisites

```
pip install customtkinter
```

All other dependencies (numpy, matplotlib, scipy) should already be installed from the analysis code.

5.2 Launching the GUI

```
cd "2_SBTDD Data Analysis Code"
python sweep_study_gui.py
```

5.3 GUI Architecture

The application is built on three core concepts:

1. **Widgets:** Visual elements (buttons, text fields, dropdowns, labels). Each is a Python object you create and place on the window.
2. **Layout (Grid):** Widgets are placed in a grid (row, column). Think of the window as a spreadsheet.
3. **Callbacks:** Functions triggered by user actions. E.g., clicking “Browse” opens a file dialog.

The basic pattern for every widget:

```
widget = SomeWidget(parent, options...)      # create
widget.grid(row=R, column=C)                 # place
widget.configure(command=my_function)         # connect callback
```

5.4 Application Structure

The GUI is a single class (SweepStudyApp) inheriting from `customtkinter.CTk`:

Table 15: GUI layout structure.

Region	Widget	Purpose
Left sidebar	CTkFrame	Study type selector, theme toggle
Center (Tab 1)	“Experimental Data”	CSV file selection, sensor parameters, pressure conversion settings
Center (Tab 2)	“CFD Comparison”	Tecplot file, domain trim bounds, CFD unit settings
Center (Tab 3)	“Results”	Embedded matplotlib plot area
Bottom	Status bar	Feedback messages

5.5 How to Extend the GUI

Adding a New Input Field

To add a new parameter (e.g., a custom title for the plot):

```
1 # Inside _build_exp_tab(), add these lines:
2 ctk.CTkLabel(tab, text="Plot Title:").grid(
3     row=row, column=0, padx=10, pady=5, sticky="w")
4 self.plot_title_var = ctk.StringVar(value="My Plot")
5 ctk.CTkEntry(tab, textvariable=self.plot_title_var, width=300).grid(
6     row=row, column=1, padx=5, pady=5, sticky="w")
7 row += 1
```

Listing 11: Adding a new text entry to a tab.

Then read the value in the callback:

```
title = self.plot_title_var.get()
```

Adding a New Tab

```
1 # In _create_main_area():
2 self.tab_new = self.tabview.add("Batch Processing")
3
4 # Then create a builder method:
5 def _build_batch_tab(self):
6     tab = self.tab_new
7     # ... add widgets here ...
```

Listing 12: Adding a new tab.

Adding a New Button with a Callback

```
1 def _build_my_section(self):
2     ctk.CTkButton(
3         tab,
4         text="Export to CSV",
5         command=self._export_results # <-- function called on click
6     ).grid(row=row, column=0, pady=10)
7
8 def _export_results(self):
9     """This runs when the user clicks 'Export to CSV'."""
10    # ... do work here ...
11    self._set_status("Exported successfully!")
```

Listing 13: Button with callback pattern.

Embedding a Matplotlib Plot

The key technique for scientific GUIs:

```
1 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
2
3 # 1. Create figure (do NOT call plt.show())
4 fig, ax = plt.subplots(figsize=(8, 5))
5 ax.plot(x, y)
6
7 # 2. Wrap in canvas and attach to a frame
8 canvas = FigureCanvasTkAgg(fig, master=self.plot_frame)
9 canvas.draw()
10 canvas.get_tk_widget().pack(fill="both", expand=True)
```

Listing 14: Embedding matplotlib inside tkinter.

5.6 User Workflow

1. Launch the app (python sweep_study_gui.py).
2. In the **Experimental Data** tab:
 - (a) Click “Browse” and select your CSV file.
 - (b) Set tap spacing, skip channels, and pressure conversion parameters.
 - (c) Choose your desired output unit from the dropdown.
 - (d) Click “Process Experimental Data”.
3. View the error-bar plot in the **Results** tab.
4. (Optional) In the **CFD Comparison** tab:
 - (a) Browse for the mcf_d_tec.bin file.
 - (b) Set the x -trim bounds and pressure offset.
 - (c) Choose a comparison unit or leave on “Auto”.
 - (d) Click “Compare Experiment vs CFD”.
5. The overlay plot appears in the **Results** tab.